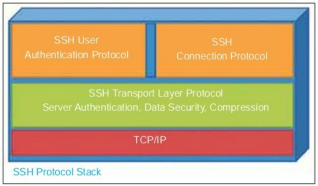

Figure 1: The SSH protocol stack

Table 2

Server configuration file	*/etc/ssh/sshd_config*
Client configuration file	*/etc/ssh/ssh_config*
Lists all hosts to be allowed to connect	*/etc/hosts.allow*
Lists all hosts to be denied	*/etc/hosts.deny*
Presence of this file refuses all users to connect except root	*/etc/nologin*

Securing the SSH service

Now let's look at how we can use the configuration information from the table, to secure the daemon service. Let's go through each relevant setting and figure out what values they need to be set to, in order to achieve the desired level of security.

Targeting defaults is the first line of defence. It is a common practice to change the default SSH port, since this makes it hard for attackers to guess the port at which service is running. Though it doesn't necessarily stop attacks, it simply makes the attackers' life difficult and hence acts as an important *config* setting. Disabling root access using passwords should be avoided; instead, the root user should be configured to accept cryptographic authentication only. Similarly, access to all users should be disabled by default, allowing only SSH protocol version 2 to be configured. The following steps in the *sshd_config* file perform these settings:

```
# only use protocol version 2
Protocol 2
# Listen on port 88000 instead of default 22
Port 88000
# Whitelist only the allowed users
AllowUsers prashant rajesh
#Optionally deny specific users
DenyUsers mayur
# No root login
PermitRootLogin no
```

Once these defaults are set up, the next step is to lock down the SSH service at the network level. The correct thing to do is to allow the SSH service to listen to only a particular IP address or a range in terms of the subnet. It is also advisable to set the IP of the machine to an address to which the SSH service should be bound. So, in short, we are setting up the 'to'and 'from' address for the SSH service's network traffic, to ensure no other IP addresses participate in the secure shell communication process. The former setting is to be done in the *sshd_config* file, while the latter is in the *hosts.allow* file, and is also called TCP wrapping.

```
#Listen only on 1.5 local ip address
ListenAddress 192.168.1.5
#Allow connections only from these ip
sshd : 10.0.0.1
```

Once the network boundaries are set, the next step is to control hosts and authentication methods. You need to ensure that empty passwords are not allowed for any user. In case of a better security set-up, password authentication should be disabled altogether, and only the public and private key pair-based authentication scheme should be in place. Depending on the security situation, sysadmins might want to disable host-based authentication and the remote hosts. Besides, when the users log in, they should be presented with a text banner, which is usually done from the compliance policy enforcement. Prior to the user login, it is possible to set up a time limit, for which the SSHD service should wait for the user to actually log in. Also, if the user is leaving the SSH session idle after logging in, there is a mechanism to log the session out. This is important to avoid idle sessions that attackers typically look for. For a given connection, SSH can be instructed to shut down the connection if a certain number of unsuccessful attempts are made. This helps to thwart brute force to some extent. The SSH daemon can also be instructed to perform a sanity check on the keyfiles and SSH directories, to ensure these are not tampered with.

```
# To enable empty passwords, change to yes (NOT RECOMMENDED)
PermitEmptyPasswords no
# Disable the use of passwords completly, only use public/
private keys
PasswordAuthentication no
#Ignore remote hosts
IgnoreRhosts yes
#Ensure that a banner shows up
Banner /etc/issue
#Wait for user to login for 60seconds
LoginGraceTime 60
#Set idle timeout to 4minutes and
ClientAliveInterval 240
ClientAliveCountMax 0
#Set maximum auth attempts for a connection
MaxAuthTries 4
```